01

Variáveis, objetos e métodos de array

Apostila do aluno · duração da aula: 2h30

O que você vai aprender

- 1. O que é uma variável?
- 2. Tipos de dados e typeof
- 3. Objetos
- 4. Arrays de objetos
- 5. Métodos de array
- 6. Desestruturação, spread, JSON, ?. e ??

Antes de começar

Esta aula começa revisando a base de tudo, a **variável**, e termina com o jeito como os dados chegam de verdade: **listas de objetos**. Os métodos de array (`map`, `filter`...) recebem funções como callback: é o mesmo mecanismo do `forEach` da aula de laços. Todos os exemplos e exercícios rodam no navegador em **site-alunos-nu.vercel.app**.

Prof. Walter Caciano · Greenn School

1 O que é uma variável?

Uma variável é uma **caixa com etiqueta**: a etiqueta é o **nome** e dentro fica um **valor**. Você cria a caixa uma vez e depois usa o nome para ler ou trocar o conteúdo.

```
let idade = 20;            // let: o valor pode mudar
const escola = "Greenn";   // const: o valor fica fixo

idade = 21;                // trocou o conteúdo da caixa
idade++;                   // 22
escola = "Outra";          // ERRO: Assignment to constant variable
```

	Pode trocar o valor?	Quando usar
`const`	não	**por padrão**: a maioria dos valores não muda
`let`	sim	quando o valor precisa mudar (contadores, somas, totais)
`var`	sim	**não use**: forma antiga, ignora os blocos `{ }` e causa bugs

Nomes: em camelCase (`notaFinal`), sem espaços, sem acento e sem começar com número. Use nomes que digam o que a variável guarda.

2 Tipos de dados e `typeof`

Tipo	Exemplo	O que é
string	`"Ana"`, `` `Olá` ``	texto
number	`42`, `7.5`	número (inteiro ou decimal)
boolean	`true`, `false`	verdadeiro ou falso
undefined	`let x;`	variável criada, mas sem valor
null	`null`	"vazio de propósito"
object	`{ nome: "Ana" }`, `[1, 2]`	objetos e arrays

```
console.log(typeof "10");          // "string"  (com aspas é texto!)
console.log(typeof 10);            // "number"
console.log(typeof null);          // "object"  (bug histórico)
console.log(Array.isArray([1, 2])); // true     (array também é "object")
```

> **const não congela o conteúdo:** `const` impede trocar a caixa, mas dá para mexer **dentro** de um array ou objeto: `frutas.push("uva")` e `aluno.nome = "Bia"` funcionam numa const.

3 Objetos

Um objeto agrupa informações sobre **uma coisa** em pares **chave: valor**. Array é uma lista em ordem (acessa pelo índice); objeto é uma ficha com campos nomeados (acessa pelo nome).

```
const aluno = {
  nome: "Ana",
  idade: 20,
  notas: [8, 9, 10],
};

aluno.nome;           // "Ana"        (ponto)
aluno["idade"];       // 20           (colchetes)
aluno.notas[1];       // 9

aluno.idade = 21;     // altera
aluno.cidade = "Recife";  // adiciona
delete aluno.notas;   // remove
"cidade" in aluno;    // true: a chave existe?
```

Métodos são funções guardadas dentro do objeto (como o `console.log` e o `frutas.push`). Dentro deles, `this` é o próprio objeto:

```
const conta = {
  saldo: 100,
  depositar(valor) {
    this.saldo += valor;
  },
};
conta.depositar(50);   // saldo = 150
```

Percorrendo: `Object.keys(obj)` (chaves), `Object.values(obj)` (valores), `Object.entries(obj)` (pares) e `for (const chave in obj)`.

4 Arrays de objetos

Na vida real os dados chegam assim: uma **lista** onde cada item é um **objeto**. É o formato de quase toda API (alunos, produtos, pedidos).

```
const alunos = [
  { nome: "Ana", nota: 9 },
  { nome: "Caio", nota: 5 },
  { nome: "Bia", nota: 7 },
];

alunos[0].nome;      // "Ana"
for (const aluno of alunos) {
  console.log(`${aluno.nome}: ${aluno.nota}`);
}
```

5 Métodos de array

Todos recebem uma função (callback) que é chamada para cada item. Nenhum deles altera o array original (exceto o `sort`).

Método	O que devolve	Exemplo
`map`	array novo, **mesmo tamanho**, cada item transformado	`alunos.map((a) => a.nome)`
`filter`	array novo só com os itens que dão **true**	`alunos.filter((a) => a.nota >= 7)`
`find`	o **primeiro** item que combina (ou undefined)	`alunos.find((a) => a.nome === "Caio")`
`some`	true se **algum** combina	`alunos.some((a) => a.nota < 6)`
`every`	true se **todos** combinam	`alunos.every((a) => a.nota > 4)`
`reduce`	**um valor só** (soma, total…)	`lista.reduce((s, n) => s + n, 0)`
`includes`	true se o valor está na lista	`cores.includes("azul")`
`join`	texto com os itens juntos	`nomes.join(", ")`

Como `map` e `filter` devolvem arrays, dá para **encadear**:

```
const texto = alunos
  .filter((a) => a.nota >= 7)     // [Ana, Bia]
  .map((a) => a.nome)             // ["Ana", "Bia"]
  .join(", ");                    // "Ana, Bia"

const total = carrinho.reduce((soma, item) => soma + item.preco * item.qtd, 0);
```

6 Desestruturação, spread, JSON, `?.` e `??`

Desestruturação tira valores de dentro de um objeto (pelo nome) ou array (pela posição). **Spread** (`...`) copia e junta:

```
const { nome, nota } = aluno;            // duas variáveis de uma vez
const [primeiro, segundo] = ["ouro", "prata"];

const copia = [...lista, 4];             // cópia com um item a mais
const atualizado = { ...aluno, nota: 10 }; // cópia trocando um campo
```

JSON é o formato de texto usado para guardar e enviar dados (é assim que uma API responde):

```
const texto = JSON.stringify(aluno);   // objeto -> texto
const obj = JSON.parse(texto);         // texto -> objeto
```

?. lê uma propriedade só se o que vem antes existir (sem erro). **??** dá um valor padrão quando o valor é null ou undefined:

```
aluno.endereco?.cidade;                  // undefined, sem erro
aluno.endereco?.cidade ?? "Sem cidade";  // valor padrão
```

7 Exercícios

Do 1 ao 2 você revisa variáveis; do 3 ao 6 trabalha com objetos; do 7 ao 12 usa métodos de array, desestruturação e spread. Em cada um, compare a sua saída com a esperada — cada letra, espaço e acento conta.

01 Variáveis — NÍVEL 1

Crie `const nome = "Ana"` e `let idade = 20`, aumente a idade com `++` e mostre a frase.

Saída esperada: Ana tem 21 anos

02 Tipos de dados — NÍVEL 1

Mostre o `typeof` de "10", 10, false e de uma variável sem valor, e `Array.isArray` de uma lista.

Saída esperada: string / number / boolean / undefined / true

03 Primeiro objeto — NÍVEL 1

Crie o objeto produto (Caderno, preço 15, estoque 3) e mostre o preço e o estoque.

Saída esperada: Caderno custa R$ 15 / Estoque: 3

04 Alterando o objeto — NÍVEL 2

Aumente o preço em 10%, adicione a categoria "papelaria" e remova o estoque com `delete`.

Saída esperada: Preço: 16.50 / Categoria: papelaria / false

05 Método e this — NÍVEL 2

Crie os métodos `depositar` e `sacar` na conta (sem deixar o saldo negativo). Deposite 50, tente sacar 500, saque 30.

Saída esperada: Saldo insuficiente / Saldo: 120

06 Percorrendo o objeto — NÍVEL 2

Use `for...in` para mostrar cada campo da pessoa no formato chave: valor.

Saída esperada: nome: Ana / idade: 20 / cidade: Recife

07 map — NÍVEL 2

Crie uma lista com os nomes dos alunos em maiúsculas (`toUpperCase`) e mostre com `join(", ")`.

Saída esperada: ANA, CAIO, BIA

08 filter

NÍVEL 3

Filtre os aprovados (nota 7 ou mais), mostre o nome de cada um e o total.

Saída esperada: Ana / Bia / Total: 2

09 find

NÍVEL 3

Ache o Caio com `find` e mostre a nota; procure o Léo e use `??` para mostrar "Não encontrado".

Saída esperada: Caio tirou 5 / Não encontrado

10 reduce

NÍVEL 3

Calcule o total do carrinho (preço × quantidade de cada item) com `reduce`.

Saída esperada: Total: R$ 140.40

11 Desestruturação e spread

NÍVEL 3

Desestruture nome e cidade, e crie uma cópia com outra cidade sem alterar o original.

Saída esperada: Ana mora em Recife / Original: Recife / Cópia: Salvador

12 Relatório da turma

DESAFIO

Mostre a média (reduce), os aprovados com nota (filter + map + join) e se todos passaram (every).

Saída esperada: Média: 7.0 / Aprovados: Ana (9), Bia (7) / Todos aprovados? false

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude um valor por vez para entender o que cada parte faz.